

Lab 2 — Change fares and retrain

MLOps workshop · Instructor draft · 20 September 2026

Hands-on: about 15 minutes, plus training/deployment wait time.

Draft, not a live-tested procedure. Future participant actions only; no existing repository or platform was changed. The instructor must pilot this flow before distribution.

Goal: simulate a 20% fare increase in the training data → commit and push → automatically retrain/deploy → compare predictions from the same endpoint.

Before starting

- Finish Lab 1 with an **accepted, deployed model**. Keep its triggers, seeded split, and model settings. Use the same clone, **main** branch, and assigned sandbox endpoint; no overlapping runs or other writers to this workspace.
- Python 3 must already work as `python` in the VS Code PowerShell terminal. Only the standard library is used. If Python is unavailable, ask the instructor.
- This is a **synthetic fare-policy scenario**, not a real-label correction. Never apply it to production data. File links are references; edit your own clone.

1. Record the Lab 1 baseline

In **AML Studio** → **Endpoints** → **Online** → **your endpoint** → **Test**, send the unchanged `data/test-request.json`. Record the two fares in order, the deployed model version, and the current `taxi-data` version.

2. Change only the training target

Open `data/taxi-data.csv`. Change only its `cost` target, not features, headers, row order, or the unnamed index column. Leave the request and batch CSV unchanged.

From the **repository root**, paste this into VS Code **PowerShell once**. It reads all rows before writing and checks the first fare to prevent repeating the increase.

```
@'
import csv
from decimal import Decimal
from pathlib import Path
path = Path("data/taxi-data.csv")
with path.open(encoding="utf-8", newline="") as stream:
    reader = csv.DictReader(stream)
    fields = reader.fieldnames
    rows = list(reader)
if not rows or Decimal(rows[0]["cost"]) != Decimal("4.5"):
    raise ValueError("Expected original fares. Do not apply twice.")
for row in rows:
    fare = Decimal(row["cost"]) * Decimal("1.20")
    row["cost"] = str(fare.quantize(Decimal("0.01")))
with path.open("w", encoding="utf-8", newline="") as stream:
    writer = csv.DictWriter(stream, fieldnames=fields)
    writer.writeheader()
    writer.writerows(rows)
print(f"Updated {len(rows)} rows; first fare: {rows[0]['cost']}")
'@ | python -
```

Checkpoint: first fares **4.5** → **5.40** and **6.0** → **7.20**; same row count, IDs, features, and header beginning `,cost,distance`. Review the diff. If the starting data differs or the script fails, stop for instructor review; do not rerun blindly.

3. Commit the data and let CI do the work

Keep `skipDataRegistration` **default: false**. The existing `mlops/azureml/train/data.yml` points to this CSV; registration must upload its changed contents. Keep that path unchanged.

In **Source Control**, stage **only the training CSV**. Commit **lab2: simulate a 20 percent fare increase**, then **Push** through the approved main-branch workflow. No `[skip ci]`, duplicate manual runs, or hyperparameter edits.

Expected chain: push → training pipeline → new data asset version → prep/train/evaluate/register → online pipeline completion trigger → deployment smoke test.

4. Verify the data and model lineage

1. **Azure DevOps:** the automatic training run uses your data-change commit.
2. **AML Studio** → **Data** → **taxi-data**: a **new version** contains the edited fares. Confirm the matching AML job's input used that version. A local edit alone does not update AML.
3. **AML Studio** → **Jobs**: all four child jobs complete. Check metrics and `deploy flag`; historical models are compared against this run's new test targets.
4. **AML Studio** → **Models**: a **new taxi-model version** belongs to this run. Online deployment starts automatically from training completion, using the same application commit.
5. **AML Studio** → **Endpoints**: `taxi-online-dp` is **Succeeded**, serves that accepted version, and has **100% traffic** after its smoke test. Registration alone is not deployment.

If rejected: the completion trigger may redeploy the old latest model. Record **not promoted** and stop for instructor review. Keep the gate and previous model; never force acceptance.

5. Consume and compare

In the same endpoint's **Test** tab, send the **identical full request** from step 1. Save the two numeric fares. **Consume** still shows the same scoring URI: the **model version changes, not the endpoint URL**. Never copy endpoint keys into evidence or source.

Input trip	Lab 1 prediction	Lab 2 prediction	Observed change
Request row 1	_____	_____	_____ %
Request row 2	_____	_____	_____ %

For a nonzero old fare, calculate $100 \times (\text{new fare} / \text{old fare} - 1)$. With the same row order, seeded split, settings, and request, expect fares **roughly 20% higher**. Floating-point split ties can change exact results; neither exact predictions nor promotion are guaranteed.

Pass: connect the commit → new data version → accepted model → deployed version → successful two-row response and comparison. If fares are unchanged, check those versions and the promotion result before editing again.

6. Explain and finish

Discuss: why did unchanged code produce different fares? Why is a green run not proof of promotion? A changed target policy does **not** prove higher accuracy; cross-run RMSE values are not automatically comparable.

Hand in: commit SHA, both pipeline run URLs, AML job URL, old/new data and model versions, and the table. Keep evidence outside the training repository to avoid another CI run.

Finish safely: delete or resize nothing. Ask the instructor whether to retain the demo fares and triggers. Pushing a revert triggers another cycle while CI is enabled. Online compute remains billable while deployed.

Reference: [Azure ML data assets and versions](#). For large/real datasets, use governed ingestion rather than routine CSV commits.